DEPARTMENT OF COMPUTER ENGINEERING

DEVOPS & CONTINUOUS TESTING LABORATORY

ACADEMIC LABORATORY MANUAL & PROJECT DOSSIER

SELENIUM TESTING PIPELINE FOR A VACCINATION REMINDER PORTAL (VAXGUARD)

End-to-End DevOps Automation: Git, Jenkins CI/CD, Docker Containerization & Ansible Provisioning

Candidate Name:	**GAYATRI PATIL**
Student ID / PRN:	**23102A0016**
Roll Number:	**16**
Course / Semester:	B.Tech / B.E. Computer Engineering (Sem VI)
Academic Year:	2026 – 2027

Certified under National Immunization Schedule • Software Quality Assurance • DevOps Capstone

CERTIFICATE OF COMPLETION

This is to certify that Miss **GAYATRI PATIL**, bearing Student ID / PRN **23102A0016** and Roll Number **16**, of Third Year Computer Engineering, has satisfactorily completed all fifteen (15) prescribed laboratory assignments and deliverables for the course of **DevOps Laboratory** entitled *'Selenium Testing Pipeline for a Vaccination Reminder Portal (VaxGuard)'* during the academic year 2026–2027.

The project work demonstrates proficient mastery of GitFlow branching, automated Selenium WebDriver regression testing, Jenkins continuous integration pipelines, multi-stage Docker containerization, Ansible idempotent infrastructure provisioning, and automated rollback mechanisms.

Internal Examiner	***External Examiner***	***Head of Department***

INDEX OF 15 DEVOPS LABORATORY ASSIGNMENTS

Task #	Deliverable Assignment Title	Status	Sign
Task 01	Problem Definition, Stakeholder Analysis and Frozen Scope	Completed	Verified
Task 02	Agile Planning, User Stories, Product Backlog and DevOps Workflow	Completed	Verified
Task 03	Requirements, Multi-Tier Architecture and Spring Boot Tech Setup	Completed	Verified
Task 04	Git and GitHub Repository Initialization, .gitignore and Branch Policy	Completed	Verified
Task 05	Feature Development with Branching: National Immunization Schedule Engine	Completed	Verified
Task 06	MVP Completion, Simulated Merge Conflict Resolution and Release Tag v1.0.0	Completed	Verified
Task 07	Jenkins Installation, SCM Polling Trigger and Continuous Integration Job	Completed	Verified
Task 08	Pipeline as Code: Declarative 8-Stage Jenkinsfile with Environment Parameters	Completed	Verified
Task 09	Selenium Automated Test Design: 5 Critical User Journeys and Screenshot Listener	Completed	Verified
Task 10	Continuous Testing in Jenkins: Quality Gate Enforcement, Defect Fix & Rerun	Completed	Verified
Task 11	Docker Multi-Stage Image Build, Non-Root Hardening and Container Lifecycle	Completed	Verified
Task 12	Jenkins-Docker Continuous Deployment: Container Rollout and Health Gate	Completed	Verified
Task 13	Configuration Management Script: Server Prerequisites & Ansible Playbook	Completed	Verified
Task 14	Automated Provisioning, Idempotency Validation (changed=0) and Fast Rollback	Completed	Verified
Task 15	Final Release Release Dossier, Live Demonstration Guide and Complete Viva Prep	Completed	Verified

Task 01: Problem Definition and Scope

DevOps Laboratory Assignment & Viva Voce Document

Class / Semester: TE Computer Engineering / DevOps Lab

Project Title: Selenium Testing Pipeline for a Vaccination Reminder Portal (VaxGuard)

---

1. Aim of the Experiment

To study the real-time clinical and operational need for an automated Vaccination Reminder Portal, identify target users, analyze existing healthcare pain points, identify key stakeholders, outline technical/operational constraints, and freeze a 15-task Minimum Viable Product (MVP) scope.

---

2. Syllabus Deliverable Requirement

> *"*"Problem statement, stakeholder list, objectives, constraints and approved MVP scope."*"*

---

3. Step-by-Step Guide: What to Show the Teacher

Step 1: Explain the Problem Definition to Teacher

- **Speech Script:**

> *"*"Madam/Sir, in Task 1, I studied the real-world challenge of childhood immunization dropouts. In physical card-based systems, parents frequently miss subsequent booster shots (such as Pentavalent-2, Measles, and DPT boosters) due to the long intervals between doses. I created the scope for **VaxGuard**, an automated web portal that calculates the complete immunization calendar based on Date of Birth and sends proactive reminders."*"*

Step 2: Show the Stakeholder Analysis Table

Show the following stakeholder matrix on your screen or in print:

Stakeholder	Category	Core Responsibilities & Needs
Parents & Caregivers	Primary End-Users	Check child's upcoming vaccine milestones, receive automated SMS/Email reminders, view historical vaccination records.
Healthcare Staff & Nurses	Clinical Users	Update vaccine status from `SCHEDULED` to `COMPLETED`, record administering nurse name and vaccine batch/lot numbers.
Public Health Administrators	System Supervisors	Monitor overall clinic immunization compliance %, identify overdue children in the catchment area, dispatch batch reminders.
DevOps & QA Engineers	Technical Maintainers	Automate end-to-end regression testing with Selenium WebDriver, enforce Jenkins CI/CD quality gates, automate container rollout.

Step 3: Show the Objectives & Measurable Success Criteria

1. **Automated Schedule Calculation:** 100% adherence to national immunization guidelines computed automatically from DOB.
2. **Automated Quality Gate:** Continuous testing in Jenkins that catches defects in < 90 seconds.
3. **Containerized Deployment:** Multi-stage Docker container deployed with non-root security.
4. **Reliability & Idempotency:** Ansible server provisioning with zero drift (`changed=0` on re-runs).

Step 4: Show the Approved 15-Task Frozen Scope

Point out the 15 sequential milestones:

1. Problem Definition and Scope
2. Agile Planning and DevOps Workflow
3. Requirements, Architecture and Technology Setup
4. Git and GitHub Repository Initialization
5. Feature Development with Branching (Schedule Engine)
6. MVP Completion and Git Collaboration (Merge Conflict & Tagging v1.0.0)

7. Jenkins Installation and Continuous Integration Job
8. Pipeline as Code and Server Deployment (Jenkinsfile)
9. Selenium Test Design and Local Execution (5 Journeys)
10. Continuous Testing in Jenkins (Quality Gate & Defect Remediation)
11. Docker Image and Container Lifecycle
12. Jenkins-Docker Continuous Deployment
13. Configuration Management Script (Ansible)
14. Automated Provisioning and Reliability Validation (Idempotency & Rollback)
15. Final End-to-End Release, Documentation and Viva

---

4. Likely Viva Voce Questions & Answers for Task 1

Q1: What are the primary pain points in the existing immunization tracking system?

Answer: Physical immunization cards get lost or damaged; parents lack automated alerts; healthcare centers cannot easily filter overdue children; and there is a high dropout rate between birth doses and multi-month boosters.

Q2: What is an MVP (Minimum Viable Product)?

Answer: An MVP is the core version of a product containing only essential features needed to deliver immediate value to users and validate hypotheses with minimal development effort. For VaxGuard, the MVP covers registration, automated schedule calculation, role-based status updates, search/filter, and a KPI dashboard.

Q3: How does DevOps add value to a healthcare application?

Answer: Healthcare applications require near-zero error tolerance. DevOps ensures reliability through automated Selenium regression testing on every commit, automated container deployment, zero-downtime rollouts, and instant automated rollback if an issue occurs.

Task 02: Agile Planning and DevOps Workflow

DevOps Laboratory Assignment & Viva Voce Document

Class / Semester: TE Computer Engineering / DevOps Lab

Project Title: Selenium Testing Pipeline for a Vaccination Reminder Portal (VaxGuard)

---

1. Aim of the Experiment

To create user stories with clear acceptance criteria for the Vaccination Reminder Portal, prepare a prioritized product backlog, construct a 15-task Kanban/Scrum plan, establish a Definition of Done (DoD), and design an end-to-end DevOps lifecycle diagram.

---

2. Syllabus Deliverable Requirement

> *“*"Backlog, task board, sprint plan, Definition of Done and DevOps workflow diagram."*”*

---

3. Step-by-Step Guide: What to Show the Teacher

Step 1: Explain Agile Planning to Teacher

- **Speech Script:**

> *“*"Madam/Sir, in Task 2, I organized the development using Agile user stories. I established clear acceptance criteria for each story, built a 15-task product backlog with story points, defined our Definition of Done (DoD), and illustrated the continuous feedback loop in our DevOps lifecycle."*”*

Step 2: Show Key User Stories with Acceptance Criteria

Show these user stories on screen:

1. **Story 1 (Child Registration & Auto Schedule):**
   - *As a* Parent / Healthcare Worker, *I want to* register a baby with Date of Birth, *so that* the system auto-calculates 16 national immunization milestones.
   - *Acceptance Criteria:* Date of Birth cannot be in the future; schedule includes BCG, HepB, OPV, Pentavalent, Rotavirus, PCV, and MR.
2. **Story 2 (Role-Based Administration):**
   - *As a* Clinical Nurse, *I want to* mark a scheduled dose as `COMPLETED`, *so that* the clinical provider name and vaccine batch/lot number are logged.
   - *Acceptance Criteria:* Parent View is read-only; Healthcare Worker mode unlocks the administration form.
3. **Story 3 (Selenium Quality Gate):**
   - *As a* DevOps Engineer, *I want* every commit to run automated Selenium tests, *so that* UI regressions are blocked before deployment.

Step 3: Show the Definition of Done (DoD)

A story is marked **Done** only when:

- [x] Code adheres to clean naming conventions and Spring Boot structure.
- [x] 100% of headless Selenium tests pass in Maven.
- [x] Jenkins build succeeds with zero compile or test errors.
- [x] Non-root multi-stage Docker container builds successfully.
- [x] Application passes `/api/health` probe with HTTP 200 OK.

Step 4: Show the DevOps Lifecycle Diagram

Show the Mermaid diagram on screen:

```
flowchart LR
    subgraph Plan_Dev [1. Plan & Code]
        A[User Stories] --> B[Feature Branch Git]
        B --> C[Local Maven Compile]
    end
    subgraph CI [2. Continuous Integration]
        D[Jenkins SCM Trigger] --> E[Selenium Automated Gate]
        E --> F[Package Executable JAR]
    end
    subgraph CD_Ops [3. Deploy & Monitor]
        G[Multi-Stage Docker] --> H[Ansible Provisioning]
        H --> I[Health Monitoring /api/health]
    end
    Plan_Dev --> CI --> CD_Ops
    CD_Ops -. Feedback .-> Plan_Dev
```

---

4. Likely Viva Voce Questions & Answers for Task 2

Q1: What is the difference between Agile and DevOps?

Answer: Agile focuses on iteratively developing and delivering software through user stories and collaboration between developers and product owners. DevOps extends Agile by bridging the gap between development and operations through automation (continuous integration, automated testing, containerization, and monitoring).

Q2: What is the difference between a Product Backlog and a Sprint Backlog?

Answer: The Product Backlog is an prioritized list of all features and requirements for the entire product. The Sprint Backlog is a specific subset of backlog items selected for completion in the current sprint.

Q3: Why is a Definition of Done (DoD) critical in DevOps?

Answer: Without a strict DoD, code might be labeled "done" when coding finishes, even if it has no automated tests or breaks during deployment. DoD ensures that testing, packaging, and health verification are completed before code is released.

Task 03: Requirements, Architecture, and Technology Setup

DevOps Laboratory Assignment & Viva Voce Document

Class / Semester: TE Computer Engineering / DevOps Lab

Project Title: Selenium Testing Pipeline for a Vaccination Reminder Portal (VaxGuard)

---

1. Aim of the Experiment

To design the minimum application architecture for the Vaccination Reminder Portal (covering create, view, update, search, role-based status workflow, and summary dashboard), select the technology stack and build tools (Java 21, Spring Boot 3.3.4, Maven), configure in-memory storage, setup deployment targets, and establish a working local development environment.

---

2. Syllabus Deliverable Requirement

> *"*"SRS summary, use-case diagram, architecture diagram, data model/API list and working local setup."*"*

---

3. Step-by-Step Guide: What to Show the Teacher

Step 1: Explain Technology Stack Selection

- **Speech Script:**

> *"*"Madam/Sir, in Task 3, I selected a modern Java 21 LTS stack with Spring Boot 3.3.4 and Maven 3.9. For database persistence, I chose an embedded in-memory H2 database, which guarantees that our entire CI/CD pipeline and Jenkins agent can build, test, and run the portal immediately without needing any external database service installed."*"*

Step 2: Show the High-Level Architecture Diagram

Show this architecture diagram on screen:

```
graph TD
    UI[Glassmorphic HTML5 / CSS3 / Vanilla JS] -->|REST API Calls| Ctrl[Spring Boot REST Controllers]
    Ctrl --> Svc[VaccinationService & ReminderService]
    Svc --> Engine[Immunization Timeline Calculator]
    Svc --> Repo[Spring Data JPA Repositories]
    Repo --> DB[(In-Memory H2 Database)]
    Ctrl --> Health[DevOps Health Gate /api/health]
```

Step 3: Show the REST API Endpoints Table

Show the core API endpoints:

- `GET /api/beneficiaries` - Retrieve registered children
- `POST /api/beneficiaries` - Register child and generate vaccination schedule
- `GET /api/vaccinations` - Filter records by status and search keyword
- `PUT /api/vaccinations/{id}/status` - Clinical administration & status update
- `POST /api/reminders/send-batch` - Dispatch simulated batch reminders
- `GET /api/dashboard/stats` - Summary KPI counts and compliance percentage
- `GET /api/health` - DevOps readiness and health verification

Step 4: Run the Terminal Verification Command

Run this in PowerShell:

```
Invoke-RestMethod -Uri "http://localhost:8080/api/health" -Method Get
```

Step 5: What Output to Point To:

Show the JSON response on screen:

```
{
  "status": "UP",
  "service": "Vaccination Reminder Portal (VaxGuard)",
  "version": "1.0.0",
  "author": "Gayatri Patil (PRN: 23102A0016, Roll: 16)",
  "database": "H2 In-Memory (Connected)",
  "qualityGate": "PASSED"
}
```

---

4. Likely Viva Voce Questions & Answers for Task 3

Q1: Why did you choose Maven over Gradle?

Answer: Maven enforces a standard project directory structure (`src/main/java`, `src/test/java`) and declarative XML configuration (`pom.xml`). It integrates natively with the Maven Surefire Plugin to produce JUnit XML reports that Jenkins consumes directly.

Q2: What is Spring Boot Actuator?

Answer: Spring Boot Actuator provides production-ready monitoring and telemetry endpoints (like `/actuator/health` and `/actuator/info`) that allow container orchestrators (Docker, Kubernetes) and CI/CD pipelines to verify application readiness without writing manual monitoring code.

Q3: How does the H2 in-memory database work?

Answer: H2 is an embedded relational database written in Java. When configured with `jdbc:h2:mem:vaxdb`, it runs inside the Java Virtual Machine (JVM) memory. It requires zero configuration, zero ports, and is destroyed and recreated automatically during test execution.

Task 04: Git and GitHub Repository Initialization

DevOps Laboratory Assignment & Viva Voce Document

Class / Semester: TE Computer Engineering / DevOps Lab

Project Title: Selenium Testing Pipeline for a Vaccination Reminder Portal (VaxGuard)

---

1. Aim of the Experiment

To initialize the Git repository for the Vaccination Reminder Portal, configure comprehensive `.gitignore` rules, establish a professional folder structure, define GitHub issue templates (bug reports and feature requests), construct a pull request template with review checklists, establish a GitFlow branch naming policy, and commit the initial application skeleton.

---

2. Syllabus Deliverable Requirement

> *"*"Repository URL, README, initial commits, issues and branch policy."*"*

---

3. Step-by-Step Guide: What to Show the Teacher

Step 1: Explain Git Setup and Branch Policy

- **Speech Script:**

> *"*"Madam/Sir, in Task 4, I set up our source control foundation following the GitFlow branching standard. I configured `.gitignore` so that generated build artifacts (like the `target/` directory and IDE metadata) are never committed. I also created issue templates for bug tracking, a pull request template, and enforced branch protection rules."*"*

Step 2: Show the Active Git Configuration

Run this command in PowerShell:

```
git status; git branch -a
```

Step 3: What Output to Point To:

Show the clean working tree and branches:

```
On branch main
nothing to commit, working tree clean
* main
  develop
```

Step 4: Show the Branch Policy Table

Show this table in [`CONTRIBUTING.md`](file:///c:/Users/ishwar/Desktop/devops%20gayatri/CONTRIBUTING.md):

Branch	Base	Purpose	Example
`main`	-	Production-ready stable release baseline.	`main` (Tagged with `v1.0.0`)
`develop`	`main`	Ongoing integration branch for tested features.	`develop`
`feature/*`	`develop`	Developing new features in isolation.	`feature/vaccination-records`
`defect/*`	`develop`	Dedicated bug fixes and remediation.	`defect/fix-registration-validation`

Step 5: Show GitHub Issue Templates

Open folder `.github/ISSUE_TEMPLATE/` and show:

- `bug_report.md` (Standardizes environment, steps to reproduce, and screenshots)
- `feature_request.md` (Standardizes problem statement and acceptance criteria)

---

4. Likely Viva Voce Questions & Answers for Task 4

Q1: Why is a `.gitignore` file necessary?

Answer: A `.gitignore` prevents unneeded, large, or security-sensitive files (such as compiled `.class` files, `.jar` packages, `.log` files, and IDE configs) from being committed into Git. This keeps the repository lightweight and prevents merge conflicts.

Q2: What is the purpose of GitFlow branching?

Answer: GitFlow provides an isolated structure: developers work on `feature/*` branches without affecting the stable `main` branch. Code is integrated into `develop` for testing, and only merged into `main` after passing quality gates.

Q3: What information should a good commit message contain?

Answer: Following the Conventional Commits specification, it includes a type (`feat`, `fix`, `test`, `ci`, `docs`), an optional scope, and a concise imperative description (e.g., `feat(workflow): implement national immunization schedule calculation`).

Task 05: Feature Development with Branching

DevOps Laboratory Assignment & Viva Voce Document

Class / Semester: TE Computer Engineering / DevOps Lab

Project Title: Selenium Testing Pipeline for a Vaccination Reminder Portal (VaxGuard)

---

1. Aim of the Experiment

To implement the first core workflow of the Vaccination Reminder Portal (child registration and national immunization schedule calculation) on a dedicated feature branch, use standard Git commands (`add`, `commit`, `push`, `pull`, `log`), raise a pull request with code review comments, and merge into the development branch using non-fast-forward merge.

---

2. Syllabus Deliverable Requirement

> *"*"Working feature 1, feature branch, pull request, review comments and merge evidence."*"*

---

3. Step-by-Step Guide: What to Show the Teacher

Step 1: Explain Feature 1 to Teacher

- **Speech Script:**

> *"*"Madam/Sir, in Task 5, I implemented our first core workflow on a dedicated branch named `feature/vaccination-records`. This feature automatically generates the full national immunization calendar when an infant is registered. I simulated raising Pull Request #1, verified the code through peer review comments, and merged it into `develop` using `git merge --no-ff`."*"*

Step 2: Show the Code Implementation

Open file: [`VaccinationService.java`](file:///c:/Users/ishwar/Desktop/devops%20gayatri/src/main/java/com/vaxguard/service/VaccinationService.java)

Point to the immunization calculation method:

```
private List<VaccinationRecord> generateImmunizationSchedule(LocalDate dob) {
    List<VaccinationRecord> records = new ArrayList<>();
    LocalDate today = LocalDate.now();

    // 1. At Birth
    records.add(createRecord("BCG", "Tuberculosis", 1, dob, today));
    records.add(createRecord("HepB-Birth", "Hepatitis B", 0, dob, today));
    records.add(createRecord("OPV-0", "Polio", 0, dob, today));

    // 2. 6 Weeks
    LocalDate sixWeeks = dob.plusWeeks(6);
    records.add(createRecord("OPV-1", "Polio", 1, sixWeeks, today));
    records.add(createRecord("Pentavalent-1", "Diphtheria, Pertussis, Tetanus, HepB, Hib", 1, sixWeeks, today));
    records.add(createRecord("Rotavirus-1", "Rotavirus Diarrhea", 1, sixWeeks, today));
    records.add(createRecord("PCV-1", "Pneumococcal Disease", 1, sixWeeks, today));
    // ... 10 weeks, 14 weeks, 9 months, 16 months
    return records;
}
```

Step 3: Run the Git Branch & Merge Evidence Command

Run this in PowerShell:

```
git log --oneline --grep="feature/vaccination-records" -n 2
```

Step 4: What Output to Point To:

Show the commit log on screen:

```
8f1233f Merge pull request #1 from feature/vaccination-records into develop
148f07f feat(workflow): implement national immunization schedule calculation and beneficiary registration
```

Step 5: Show Pull Request #1 Details

Show document: [`docs/TASK-05-Feature-Development-Branching.md`](file:///c:/Users/ishwar/Desktop/devops%20gayatri/docs/TASK-05-Feature-Development-Branching.md)

Point to:

- **Title:** `[PR #1] feat(workflow): National Immunization Schedule & Child Registration Engine`
- **Reviewer Comment:** *"Confirmed: any scheduled dose with a calendar date prior to today minus 7 days automatically receives status OVERDUE."*
- **Status:** Approved and Merged into `develop`.

---

4. Likely Viva Voce Questions & Answers for Task 5

Q1: What does `git merge --no-ff` do?

Answer: The `--no-ff` (no fast-forward) flag forces Git to create a dedicated merge commit, even if the target branch could have fast-forwarded. This preserves the historical record of the feature branch and pull request in the Git graph.

Q2: How does the application determine if a vaccination is OVERDUE?

Answer: When computing the schedule, the engine compares the vaccine's milestone date against `LocalDate.now().minusDays(7)`. If the due date is in the past by more than 7 days, its status is set to `OVERDUE`.

Q3: Why use Pull Requests in Agile/DevOps teams?

Answer: Pull Requests facilitate mandatory peer code review, automated CI testing, discussion on architectural choices, and documentation before code is merged into the shared codebase.

Task 06: MVP Completion and Git Collaboration

DevOps Laboratory Assignment & Viva Voce Document

Class / Semester: TE Computer Engineering / DevOps Lab

Project Title: Selenium Testing Pipeline for a Vaccination Reminder Portal (VaxGuard)

---

1. Aim of the Experiment

To implement the remaining essential MVP functions for the Vaccination Reminder Portal (create, view, update, search records; role-based status workflow; summary dashboard with Chart.js), demonstrate development on a second feature branch, simulate merge conflict creation and manual resolution, tag the release baseline as `v1.0.0`, and update the product backlog.

---

2. Syllabus Deliverable Requirement

> *"*"Functional MVP, resolved merge conflict, tagged version and updated backlog."*"*

---

3. Step-by-Step Guide: What to Show the Teacher

Step 1: Explain MVP Completion & Collaboration

- **Speech Script:**

> *"*"Madam/Sir, in Task 6, I completed our remaining MVP features on branch `feature/status-workflow-dashboard`. To satisfy the syllabus requirement of Git collaboration, I simulated a real-world merge conflict between two branches modifying application properties, demonstrated manual conflict resolution, merged into `main`, and tagged our release as `v1.0.0`."*"*

Step 2: Show the Git Commit Graph and Release Tag

Run this command in PowerShell:

```
git log --oneline --graph --all -n 8; git tag -n
```

Step 3: What Output to Point To:

Show the Git tree graph and release tag on screen:

```
*   b3b2fd1 chore(release): merge develop into main for MVP release v1.0.0
|\
| *   adab740 Merge pull request #2 from feature/status-workflow-dashboard into develop
| |\
| | * c64b7a9 feat(dashboard): interactive UI, role switcher, Chart.js, Selenium test suite
| |/
| *   8f1233f Merge pull request #1 from feature/vaccination-records into develop
v1.0.0          Release v1.0.0: Functional MVP of Vaccination Reminder Portal with Selenium Pipeline
```

Step 4: Explain the Merge Conflict Resolution

Show the conflict markers in [`docs/TASK-06-MVP-Completion-and-Git-Collaboration.md`](file:///c:/Users/ishwar/Desktop/devops%20gayatri/docs/TASK-06-MVP-Completion-and-Git-Collaboration.md):

```
<<<<<<< HEAD (Branch: develop)
app.default.reminder.channel=SMS_AND_EMAIL
=======
app.default.reminder.channel=WHATSAPP_PRIMARY
>>>>>>> conflict/vaccination-status-update
```

- **Explain your fix:** *"I resolved this by combining both into `MULTI_CHANNEL_SMS_EMAIL_WHATSAPP`, staged the file with `git add`, and committed the clean resolution."*

Step 5: Show the Live MVP in Browser

Open `http://localhost:8080` in Chrome/Edge and show:

- Real-time KPI summary cards (Total Beneficiaries, Completed Doses, Compliance Rate %)
- Interactive Chart.js donut chart
- Live search input & status filter dropdown (`SCHEDULED`, `COMPLETED`, `OVERDUE`)
- Role switcher (■■■■ `Health Worker` vs ■■■■■ `Parent View`)

---

4. Likely Viva Voce Questions & Answers for Task 6

Q1: What creates a Git merge conflict?

Answer: A merge conflict occurs when two branches make different changes to the same line(s) of a file, or one branch deletes a file that another branch modified. Git cannot automatically decide which change to keep and asks the developer to resolve it manually.

Q2: What is the difference between a lightweight tag and an annotated tag in Git?

Answer: A lightweight tag is simply a pointer to a specific commit. An annotated tag (created with `git tag -a`) is stored as a full object in the Git database containing the tagger name, email, date, and a release message. We used an annotated tag for `v1.0.0`.

Q3: How do role-based permissions work in your MVP?

Answer: In Parent View, the interface is read-only; clinical administration buttons are hidden. Switching to Healthcare Worker mode unlocks the `[■ Administer]` buttons to record the nurse's name, vaccine batch number, and mark doses `COMPLETED`.

Task 07: Jenkins Installation and Continuous Integration Job

DevOps Laboratory Assignment & Viva Voce Document

Class / Semester: TE Computer Engineering / DevOps Lab

Project Title: Selenium Testing Pipeline for a Vaccination Reminder Portal (VaxGuard)

---

1. Aim of the Experiment

To install and configure Jenkins Continuous Integration server, configure Global Tools (JDK 21, Maven 3.9), connect the Vaccination Reminder Portal Git repository, create an automated build job triggered by commit or scheduled polling, and archive the build artifact.

---

2. Syllabus Deliverable Requirement

> *"*"Configured Jenkins job, successful build log, trigger evidence and archived artefact."*"*

---

3. Step-by-Step Guide: What to Show the Teacher

Step 1: Explain Jenkins Setup to Teacher

- **Speech Script:**

> *"*"Madam/Sir, in Task 7, I set up Jenkins continuous integration. I defined Global Tools for Java 21 and Maven, and configured an automated build job that tracks our repository. When code is pushed or when scheduled polling detects a new commit, Jenkins automatically pulls the code, compiles it, runs test suites, and archives the resulting JAR file."*"*

Step 2: Show the Jenkins Job Configuration File

Open file: [`jenkins/job-config.xml`](file:///c:/Users/ishwar/Desktop/devops%20gayatri/jenkins/job-config.xml)

Point out:

- **Git SCM Connection:** `https://github.com/gayatri-patil/vaccination-reminder-portal.git`
- **Branch Specifier:** `*/main`
- **Polling Trigger:** `H/15 * * * *` (Every 15 minutes)
- **Pipeline Script Path:** `jenkins/Jenkinsfile`

Step 3: Show the Successful Build Log Evidence

Show the execution log in [`docs/TASK-07-Jenkins-Installation-and-Continuous-Integration-Job.md`](file:///c:/Users/ishwar/Desktop/devops%20gayatri/docs/TASK-07-Jenkins-Installation-and-Continuous-Integration-Job.md):

```
Started by an SCM change
Checking out Revision b3b2fd1 (refs/remotes/origin/main)
[Vaccination-Reminder-Portal-CI] $ cmd.exe /C "mvnw.cmd clean test package && exit %%ERRORLEVEL%%"
[INFO] Scanning for projects...
[INFO] Building vaccination-reminder-portal 1.0.0
[INFO] --- surefire:3.2.5:test (default-test) @ vaccination-reminder-portal ---
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0
[INFO] Building jar: target/vaccination-reminder-portal-1.0.0.jar
[INFO] BUILD SUCCESS
Archiving artifacts: target/*.jar
Recording test results: target/surefire-reports/*.xml
Finished: SUCCESS
```

Step 4: Show the Archived Artifact

- Point to file: [`target/vaccination-reminder-portal-1.0.0.jar`](file:///c:/Users/ishwar/Desktop/devops%20gayatri/target/)
- Explain that this is a fully packaged, self-contained Spring Boot executable fat JAR containing all embedded web server and application dependencies.

---

4. Likely Viva Voce Questions & Answers for Task 7

Q1: What is Continuous Integration (CI)?

Answer: Continuous Integration is a software development practice where developers frequently merge code changes into a central repository. An automated build server (like Jenkins) automatically compiles the code and executes automated tests on every push to detect integration errors early.

Q2: What is the difference between Webhooks and SCM Polling?

Answer: SCM Polling is pull-based: Jenkins periodically queries the Git server (e.g., every 15 minutes) to see if new commits exist. A Webhook is push-based: GitHub immediately sends an HTTP POST event to Jenkins the instant a developer pushes code, triggering the build without delay.

Q3: What does "Archiving an Artifact" mean in Jenkins?

Answer: Archiving preserves the output binaries produced during a build (such as `.jar` or `.war` files) inside Jenkins storage. This allows teams to download, audit, or deploy a specific build artifact later without having to recompile the source code.

Task 08: Pipeline as Code and Server Deployment

DevOps Laboratory Assignment & Viva Voce Document

Class / Semester: TE Computer Engineering / DevOps Lab

Project Title: Selenium Testing Pipeline for a Vaccination Reminder Portal (VaxGuard)

---

1. Aim of the Experiment

To create a declarative Jenkinsfile for the Vaccination Reminder Portal covering checkout, build, compile, Selenium automated test gate, packaging, and server deployment stages, deploy the application to Tomcat / container server, parameterize deployment environment settings (`staging`/`production`), and verify the deployed application.

---

2. Syllabus Deliverable Requirement

> *"*"Jenkinsfile, successful pipeline run, deployed application URL/screenshot and configuration evidence."*"*

---

3. Step-by-Step Guide: What to Show the Teacher

Step 1: Explain Declarative Pipeline as Code

- **Speech Script:**

> *"*"Madam/Sir, in Task 8, I codified our deployment pipeline into a declarative `Jenkinsfile` placed in the root of our repository. It defines 8 sequential stages and accepts parameters for target environment (`staging` or `production`), port numbers, and Docker image tags. It guarantees that our deployment logic is version-controlled in Git alongside our application source code."*"*

Step 2: Show the Jenkinsfile Stages

Open file: [`jenkins/Jenkinsfile`](file:///c:/Users/ishwar/Desktop/devops%20gayatri/jenkins/Jenkinsfile)

Walk through the 8 stages:

1. `Stage 1: Checkout SCM` - Pulls branch from Git
2. `Stage 2: Environment Verification` - Validates Java 21, Maven, Docker
3. `Stage 3: Compile & Static Analysis` - Clean compilation
4. `Stage 4: Selenium Automated Testing Quality Gate` - Headless test execution
5. `Stage 5: Package Application Artifact` - Generates executable JAR
6. `Stage 6: Docker Image Build & Tag` - Builds versioned container image
7. `Stage 7: Continuous Deployment` - Deploys to target environment
8. `Stage 8: Automated Health Check Verification` - Validates HTTP 200 OK

Step 3: Show the Parameterized Pipeline Settings

Point to lines 15–22 of `Jenkinsfile`:

```
parameters {
    choice(name: 'ENVIRONMENT', choices: ['staging', 'production'], description: 'Deployment Target Environment')
    string(name: 'APP_PORT', defaultValue: '8080', description: 'Application Port Mapping')
    booleanParam(name: 'RUN_SELENIUM_TESTS', defaultValue: true, description: 'Execute Selenium Automated UI Quality Gate'
    string(name: 'DOCKER_IMAGE_TAG', defaultValue: '1.0.0', description: 'Docker Release Tag')
}
```

Step 4: Show the Deployed Application URL and Health Proof

Run in PowerShell:

```
Invoke-RestMethod -Uri "http://localhost:8080/api/health" -Method Get
```

Point to the verified status:

```
{
  "status": "UP",
  "service": "Vaccination Reminder Portal (VaxGuard)",
  "version": "1.0.0",
  "author": "Gayatri Patil (PRN: 23102A0016, Roll: 16)",
  "database": "H2 In-Memory (Connected)",
  "qualityGate": "PASSED"
}
```

---

4. Likely Viva Voce Questions & Answers for Task 8

Q1: What is the difference between Scripted Pipeline and Declarative Pipeline?

Answer: Scripted Pipeline uses Groovy procedural scripting with fewer syntactic constraints. Declarative Pipeline provides a more structured, readable, and opinionated syntax with predefined sections (`pipeline`, `stages`, `steps`, `post`, `parameters`). Declarative is the modern industry standard for Jenkins pipelines.

Q2: What is the purpose of the `post` block in a Jenkinsfile?

Answer: The `post` block defines actions that execute automatically based on the stage or pipeline outcome: `always` (e.g., archiving JUnit reports and screenshots), `success` (e.g., notification alerts), or `failure` (e.g., notifying developers of a broken build).

Q3: How is environment parameterization useful?

Answer: Parameterization allows the same Jenkinsfile to deploy code to different environments (such as `staging` on port 8080 or `production` on port 80) simply by selecting a dropdown option in the Jenkins UI, without needing separate scripts.

Task 09: Selenium Test Design and Local Execution

DevOps Laboratory Assignment & Viva Voce Document

Class / Semester: TE Computer Engineering / DevOps Lab

Project Title: Selenium Testing Pipeline for a Vaccination Reminder Portal (VaxGuard)

---

1. Aim of the Experiment

To identify 3 to 5 critical user journeys in the Vaccination Reminder Portal, write automated Selenium WebDriver test cases using JUnit 5 with assertions and test fixtures, implement an automated failure screenshot capture mechanism, and execute the test suite locally through Maven.

---

2. Syllabus Deliverable Requirement

> *"*"Test plan, Selenium scripts, local test report and failure screenshot mechanism."*"*

---

3. Step-by-Step Guide: What to Show the Teacher

Step 1: Explain the Selenium Test Suite

- **Speech Script:**

> *"*"Madam/Sir, in Task 9, I designed end-to-end automated UI regression tests using **Selenium WebDriver 4** and **JUnit 5**. I mapped 5 critical user journeys. All tests are configured to run in headless mode so that they can run reliably inside Linux CI/CD agents without requiring a graphical monitor. I also wrote a custom JUnit extension that captures browser screenshots whenever an assertion fails."*"*

Step 2: Show the 5 User Journey Test Classes

Open folder: [`src/test/java/com/vaxguard/selenium/`](file:///c:/Users/ishwar/Desktop/devops%20gayatri/src/test/java/com/vaxguard/selenium/)

Walk through the 5 test journeys:

Journey #	Test Class	Core Verification & Assertions
01	`BeneficiaryRegistrationJourneyTest`	Fills child form with DOB, submits, verifies child record in registry, and confirms 16 vaccination doses were generated automatically.
02	`SearchAndFilterJourneyTest`	Searches by child name and filters records by status (`COMPLETED`, `SCHEDULED`); asserts returned rows match query.
03	`StatusWorkflowJourneyTest`	Tests clinical role workflow: nurse enters batch lot #, updates status to `COMPLETED`; asserts Parent View cannot edit records.
04	`DashboardMetricsJourneyTest`	Verifies numeric KPI cards (Total, Completed, Upcoming, Compliance Rate %) and Chart.js canvas render.
05	`DefectSimulationJourneyTest`	Quality Gate: simulates defect failure, captures failure screenshot, and proves pass upon bugfix.

Step 3: Show the Failure Screenshot Mechanism

Open file: [`ScreenshotOnFailureExtension.java`](file:///c:/Users/ishwar/Desktop/devops%20gayatri/src/test/java/com/vaxguard/selenium/ScreenshotOnFailureExtension.java)

Explain how it captures screenshots:

```
public class ScreenshotOnFailureExtension implements TestWatcher {
    @Override
    public void testFailed(ExtensionContext context, Throwable cause) {
        WebDriver driver = ((BaseSeleniumTest) testInstance).getDriver();
        File screenshot = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);
        String fileName = String.format("%s_%s_FAILURE_%s.png", className, methodName, timestamp);
        Files.copy(screenshot.toPath(), Paths.get("target/screenshots").resolve(fileName));
    }
}
```

Step 4: Run the Maven Test Compile Verification

Run in PowerShell:

```
.\mvnw.cmd test-compile
```

Show: `BUILD SUCCESS` with all 8 test source files compiled cleanly.

---

4. Likely Viva Voce Questions & Answers for Task 9

Q1: What is Headless Browser Testing?

Answer: A headless browser executes web pages without displaying a graphical user interface (GUI). It runs entirely in background memory. In DevOps CI/CD pipelines (like Jenkins running on Linux servers without a desktop environment), headless mode is mandatory for running browser automation.

Q2: What is the difference between Implicit Wait and Explicit Wait in Selenium?

Answer: Implicit Wait tells WebDriver to poll the DOM for a specified period (e.g., 5 seconds) when looking for any element. Explicit Wait (`WebDriverWait` and `ExpectedConditions`) pauses execution until a specific condition is met (e.g., element becomes clickable or visible) before proceeding, avoiding arbitrary thread sleeps.

Q3: Why use Page Object Model (POM) or Base Test classes?

Answer: In our suite, `BaseSeleniumTest` abstracts browser setup (auto-detecting Chrome or Edge, configuring headless mode), server port injection, and teardown logic. This keeps individual test classes clean, reusable, and maintainable.

Task 10: Continuous Testing in Jenkins (Quality Gate & Defect Remediation)

DevOps Laboratory Assignment & Viva Voce Document

Class / Semester: TE Computer Engineering / DevOps Lab

Project Title: Selenium Testing Pipeline for a Vaccination Reminder Portal (VaxGuard)

---

1. Aim of the Experiment

To integrate the Vaccination Reminder Portal Selenium test suite into the Jenkins pipeline as an automated Quality Gate, publish JUnit test reports, configure the pipeline so failed tests halt packaging and deployment, deliberately introduce a software defect, demonstrate build failure and screenshot capture, fix the defect through a code commit, and rerun the pipeline successfully.

---

2. Syllabus Deliverable Requirement

> *"*"Jenkins test report, failed pipeline evidence, defect correction commit and successful rerun."*"*

---

3. Step-by-Step Guide: What to Show the Teacher

Step 1: Explain the Quality Gate Concept

- **Speech Script:**

> *"*"Madam/Sir, Task 10 demonstrates our automated Quality Gate in Jenkins. The syllabus requires demonstrating that a deliberately introduced defect causes the pipeline to fail and capture a screenshot, and then showing that fixing the defect allows the pipeline to pass. I created an intentional defect toggle in the registration engine to prove this exact quality gate behavior."*"*

Step 2: Show the Defect Simulation Code

Open file: [`DefectSimulationJourneyTest.java`](file:///c:/Users/ishwar/Desktop/devops%20gayatri/src/test/java/com/vaxguard/selenium/DefectSimulationJourneyTest.java)

Explain how the test works:

- **Phase 1 (Defect Introduced):** When `defectModeEnabled = true`, child registration throws an intentional HTTP 400 validation error. The Selenium assertion fails because the expected success toast does not appear.
- **Phase 2 (Screenshot Captured):** The `ScreenshotOnFailureExtension` captures a PNG snapshot of the broken UI into `target/screenshots/`.
- **Phase 3 (Pipeline Aborted):** Jenkins skips the Package, Docker Build, and Deploy stages, protecting production from being contaminated.
- **Phase 4 (Remediation Commit):** Resetting `defectModeEnabled = false` simulates the bugfix commit. The test re-runs and passes cleanly.

Step 3: Show the UI Defect Toggle in the Live Portal

1. Open in browser: `http://localhost:8080`
2. Click the navigation tab: **"■ DevOps Pipeline & Quality Gate"**
3. Point out the interactive card:
   - *"Task 10: Intentional Defect Injection & Quality Gate Demonstration"*
   - Checkbox: `[ ] Enable Simulated Defect`
   - Real-time status badge: `Status: System Normal (Defect Disabled - All Quality Gates Pass)`

Step 4: Show the Failed Build Evidence Log in Documentation

Open [`docs/TASK-10-Continuous-Testing-Jenkins-Quality-Gate.md`](file:///c:/Users/ishwar/Desktop/devops%20gayatri/docs/TASK-10-Continuous-Testing-Jenkins-Quality-Gate.md) and point out:

- Failure output: `[ERROR] AssertionFailedError: Success toast message should be displayed ==> expected: but was:`
- Screenshot capture notice: `[SELENIUM QUALITY GATE] Test failure screenshot captured: target/screenshots/BeneficiaryRegistrationJourneyTest_FAILURE.png`
- Skipped downstream stages: `stage (5. Package) - SKIPPED`, `stage (6. Docker) - SKIPPED`, `stage (7. Deploy) - SKIPPED`.
- Successful rerun log showing `Finished: SUCCESS`.

---

4. Likely Viva Voce Questions & Answers for Task 10

Q1: What is a Quality Gate in a CI/CD pipeline?

Answer: A Quality Gate is an automated enforcement checkpoint. In our pipeline, it ensures that code can only progress to packaging and deployment if 100% of the automated Selenium tests pass without failures or errors.

Q2: What happens if a test fails in Jenkins?

Answer: The Maven Surefire plugin records the test failure in an XML report, the failure screenshot listener saves a browser screenshot, and Jenkins marks the stage as FAILED. Using Jenkins pipeline logic, the pipeline halts immediately, preventing defective code from reaching staging or production.

Q3: How does automated screenshot capture assist developers?

Answer: Unlike text logs or stack traces, a screenshot provides immediate visual evidence of the exact state of the web page at the moment of failure (e.g., unexpected modal error, missing button, or broken CSS layout), drastically reducing debugging time.

Task 11: Docker Image and Container Lifecycle

DevOps Laboratory Assignment & Viva Voce Document

Class / Semester: TE Computer Engineering / DevOps Lab

Project Title: Selenium Testing Pipeline for a Vaccination Reminder Portal (VaxGuard)

---

1. Aim of the Experiment

To create a multi-stage Dockerfile for the Vaccination Reminder Portal, build and tag the Docker image (`vaxguard/vaccination-portal:1.0.0`), run the container with port mapping (`8080:8080`), inspect runtime logs, verify container health, execute the complete container lifecycle (run, stop, start, restart, remove), and document the lifecycle commands.

---

2. Syllabus Deliverable Requirement

> *"*"Dockerfile, image details, Docker command log and running container evidence."*"*

---

3. Step-by-Step Guide: What to Show the Teacher

Step 1: Explain the Multi-Stage Dockerfile Architecture

- **Speech Script:**

> *"*"Madam/Sir, in Task 11, I containerized VaxGuard using a production-ready multi-stage Dockerfile. Stage 1 compiles the application using Eclipse Temurin JDK 21 and Maven. Stage 2 copies only the packaged fat JAR into a minimal Eclipse Temurin JRE 21 Alpine image. I secured the container by running under an unprivileged user `vaxuser` rather than root, and configured an automated health check."*"*

Step 2: Show the Dockerfile Code

Open file: [`docker/Dockerfile`](file:///c:/Users/ishwar/Desktop/devops%20gayatri/docker/Dockerfile)

Point out key production lines:

```
# STAGE 1: Builder
FROM eclipse-temurin:21-jdk-alpine AS builder
WORKDIR /build
COPY pom.xml mvnw ./
COPY src ./src
RUN ./mvnw clean package -DskipTests

# STAGE 2: Lightweight Runtime
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
# Non-root user creation
RUN addgroup -S vaxgroup && adduser -S vaxuser -G vaxgroup
COPY --from=builder /build/target/vaccination-reminder-portal-1.0.0.jar app.jar
RUN chown -R vaxuser:vaxgroup /app
USER vaxuser

EXPOSE 8080

HEALTHCHECK --interval=20s --timeout=5s --retries=3 \
  CMD wget --spider http://localhost:8080/api/health || exit 1

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Step 3: Show the Complete Container Lifecycle Script

Open file: [`docker/container-lifecycle.ps1`](file:///c:/Users/ishwar/Desktop/devops%20gayatri/docker/container-lifecycle.ps1)

Explain the 8 lifecycle commands demonstrated:

1. `docker build -t vaxguard/vaccination-portal:1.0.0 -f docker/Dockerfile .` (Builds image)
2. `docker tag vaxguard/vaccination-portal:1.0.0 vaxguard/vaccination-portal:latest` (Tags latest)
3. `docker images vaxguard/vaccination-portal` (Lists local image details: ~214MB)
4. `docker run -d --name vaxguard-container -p 8080:8080 vaxguard/vaccination-portal:1.0.0` (Runs with port mapping)
5. `docker ps --filter "name=vaxguard-container"` (Inspects container status: `Up (healthy)`)

6. `docker logs --tail 20 vaxguard-container` (Inspects Spring Boot startup logs)
7. `docker stop vaxguard-container` (Gracefully stops container)
8. `docker start vaxguard-container` (Restarts container)

---

4. Likely Viva Voce Questions & Answers for Task 11

Q1: What are the advantages of a multi-stage Docker build?

Answer: Multi-stage builds keep intermediate build dependencies (like JDK 21 compiler, Maven plugins, and source code) out of the final runtime image. This dramatically shrinks image size (from ~800MB to ~214MB) and hardens container security by eliminating compilers and debuggers from production.

Q2: Why should containers not run as the root user?

Answer: If an attacker exploits an application vulnerability inside a container running as root, they could potentially escape the container and gain root administrative access over the host machine. Creating a dedicated non-root user (`vaxuser:vaxgroup`) prevents this privilege escalation.

Q3: What does Docker port mapping (`-p 8080:8080`) do?

Answer: The `-p :` flag forwards network traffic hitting port 8080 on the host machine to port 8080 inside the container's isolated network namespace, allowing external browsers to access the portal.

Task 12: Jenkins-Docker Continuous Deployment

DevOps Laboratory Assignment & Viva Voce Document

Class / Semester: TE Computer Engineering / DevOps Lab

Project Title: Selenium Testing Pipeline for a Vaccination Reminder Portal (VaxGuard)

---

1. Aim of the Experiment

To extend the Vaccination Reminder Portal Jenkins pipeline to build a versioned Docker image, publish/tag it locally or to a container registry, deploy a fresh container automatically after successful Selenium tests, and verify continuous end-to-end commit-to-container deployment.

---

2. Syllabus Deliverable Requirement

> *"*"Versioned image, registry evidence and end-to-end commit-to-container pipeline."*"*

---

3. Step-by-Step Guide: What to Show the Teacher

Step 1: Explain the Commit-to-Container Pipeline

- **Speech Script:**

> *"*"Madam/Sir, in Task 12, I extended our Jenkins pipeline to achieve Continuous Deployment (CD). As soon as the Selenium tests pass in Stage 4, Stage 6 automatically builds a versioned Docker image (`vaxguard/vaccination-portal:1.0.0`). Stage 7 safely replaces the existing container with the new version on port 8080, and Stage 8 performs an automated health check against `/api/health`."*"*

Step 2: Show the Docker Deployment Pipeline Stages

Open file: [`jenkins/Jenkinsfile`](file:///c:/Users/ishwar/Desktop/devops%20gayatri/jenkins/Jenkinsfile)

Point to lines 75–115:

```
stage('6. Docker Image Build & Tag') {
    steps {
        echo "=== Stage 6: Building Versioned Docker Container Image ==="
        sh "docker build -t ${DOCKER_IMAGE} -t vaxguard/vaccination-portal:latest -f docker/Dockerfile ."
    }
}

stage('7. Continuous Deployment (Docker / Server)') {
    steps {
        echo "=== Stage 7: Deploying Fresh Container to ${params.ENVIRONMENT} ==="
        script {
            sh """
                docker stop ${CONTAINER_NAME} || true
                docker rm ${CONTAINER_NAME} || true
                docker run -d --name ${CONTAINER_NAME} \
                    -p ${params.APP_PORT}:8080 \
                    -e SPRING_PROFILES_ACTIVE=${params.ENVIRONMENT} \
                    ${DOCKER_IMAGE}
            """
        }
    }
}

stage('8. Automated Health Check Verification') {
    steps {
        echo "=== Stage 8: Verifying Application Health Readiness ==="
        sleep 8
        sh "curl -f http://localhost:${params.APP_PORT}/api/health || exit 1"
    }
}
```

Step 3: Show Docker Compose Orchestration

Open file: [`docker/docker-compose.yml`](file:///c:/Users/ishwar/Desktop/devops%20gayatri/docker/docker-compose.yml)

Point out:

- Image: `vaxguard/vaccination-portal:1.0.0`

- Container Name: `vaxguard-app`
- Port Mapping: `8080:8080`
- Automated Healthcheck: `CMD wget -qO- http://localhost:8080/api/health`

Step 4: Show Versioned Image Details

Show the local image registry output in [`docs/TASK-12-Jenkins-Docker-Continuous-Deployment.md`](file:///c:/Users/ishwar/Desktop/devops%20gayatri/docs/TASK-12-Jenkins-Docker-Continuous-Deployment.md):

```
REPOSITORY                    TAG       IMAGE ID       SIZE
vaxguard/vaccination-portal   1.0.0     d4982a1b920e   214MB
vaxguard/vaccination-portal   latest    d4982a1b920e   214MB
```

---

4. Likely Viva Voce Questions & Answers for Task 12

Q1: What is the difference between Continuous Delivery and Continuous Deployment?

Answer: In Continuous Delivery, software passes automated builds and tests and is packaged into release-ready artifacts, but deployment to production requires manual human approval. In Continuous Deployment, every change that passes all automated quality gates is deployed directly to production with zero manual intervention.

Q2: How does your pipeline ensure that a broken Docker container is never deployed?

Answer: Stage 4 (Selenium Quality Gate) executes *before* Stage 6 (Docker Build) and Stage 7 (Deployment). If any Selenium test fails, the pipeline aborts immediately, preventing the Docker image from ever being built or deployed.

Q3: Why is health check verification necessary after deployment?

Answer: Just because a container starts (`docker run`) does not guarantee that the application inside is ready to accept user requests (e.g., database connection might be initializing). Stage 8 verifies HTTP 200 from `/api/health` before marking the deployment as successful.

Task 13: Configuration Management Script (Ansible)

DevOps Laboratory Assignment & Viva Voce Document

Class / Semester: TE Computer Engineering / DevOps Lab

Project Title: Selenium Testing Pipeline for a Vaccination Reminder Portal (VaxGuard)

---

1. Aim of the Experiment

To identify all server prerequisites for the Vaccination Reminder Portal (packages, users, folders, files, network ports, and system services), construct an Ansible inventory (`inventory.ini`), and develop an automated YAML playbook (`playbook.yml`) with Jinja2 systemd templates to provision the target node.

---

2. Syllabus Deliverable Requirement

> *"*"Configuration specification, Puppet manifest or Ansible playbook and first execution log."*"*

---

3. Step-by-Step Guide: What to Show the Teacher

Step 1: Explain Ansible Configuration Management

- **Speech Script:**

> *"*"Madam/Sir, in Task 13, I automated server provisioning using Ansible. I codified all prerequisite specifications—installing OpenJDK 21, Docker, and UFW firewall; creating an unprivileged system user `vaxadmin`; provisioning directory trees `/opt/vaccination-portal`; templating systemd unit files; and managing firewall ports 8080 and 22 into `ansible/playbook.yml`."*"*

Step 2: Show the Server Prerequisite Specification Table

Show this specification table on screen:

Component	Target Requirement	Purpose
Target OS	Ubuntu 22.04 LTS / Debian 12	Standard Linux host server
System Packages	`openjdk-21-jre-headless`, `docker.io`, `curl`, `git`, `ufw`	Core runtime and administration tools
Application User	`vaxadmin` (Secondary group: `docker`)	Isolated non-privileged service user
Filesystem Tree	`/opt/vaccination-portal/{releases,logs,config}`	Clean separation of releases and logs
Firewall Ports	`8080/tcp` (Web HTTP), `22/tcp` (SSH)	Security access control via UFW
Service Supervisor	`vaxguard.service` (`systemd`)	Process supervision, auto-restart on failure

Step 3: Show the Ansible Playbook

Open file: [`ansible/playbook.yml`](file:///c:/Users/ishwar/Desktop/devops%20gayatri/ansible/playbook.yml)

Walk through the 12 tasks:

1. Update APT package cache
2. Install OpenJDK 21, Docker, curl, git, ufw
3. Ensure Docker service is enabled and started
4. Create dedicated user and group `vaxadmin`
5. Create application directory hierarchy
6. Configure UFW firewall rules for ports 8080 and 22
7. Deploy application executable JAR
8. Manage symlink `/opt/vaccination-portal/app.jar`
9. Template systemd service (`templates/vaxguard.service.j2`)
10. Ensure VaxGuard systemd service is active and enabled
11. Automated health check verification
12. Output provisioning summary

Step 4: Show the First Execution Log

Open [`docs/TASK-13-Configuration-Management-Ansible.md`](file:///c:/Users/ishwar/Desktop/devops%20gayatri/docs/TASK-13-Configuration-Management-Ansible.md) and point out:

- Initial execution recap: `target-node-01 : ok=14 changed=12 unreachable=0 failed=0`
- Confirming that all 12 configuration elements were provisioned cleanly.

---

4. Likely Viva Voce Questions & Answers for Task 13

Q1: What is Configuration Management (CM)?

Answer: Configuration Management is the practice of maintaining computer systems, servers, and software in a known, desired, and consistent state using automated, version-controlled code rather than manual administration.

Q2: What is the difference between Ansible and Puppet/Chef?

Answer: Ansible is **agentless**: it uses standard SSH to connect to target nodes and does not require installing any background daemon on managed servers. Puppet and Chef are **agent-based**, requiring dedicated background client agents running on every target machine.

Q3: What is the role of the `inventory.ini` file in Ansible?

Answer: The inventory file defines the target hostnames, IP addresses, groups (e.g., `[vax_servers]`), connection credentials (SSH users and private keys), and environment variables that Ansible connects to and manages.

Task 14: Automated Provisioning and Reliability Validation

DevOps Laboratory Assignment & Viva Voce Document

Class / Semester: TE Computer Engineering / DevOps Lab

Project Title: Selenium Testing Pipeline for a Vaccination Reminder Portal (VaxGuard)

---

1. Aim of the Experiment

To provision a clean target environment using Ansible, deploy the application/container, rerun the automation to demonstrate idempotency, perform automated health checks on the running service, and demonstrate automated rollback and recovery to the previous stable release (`v1.0.0`).

---

2. Syllabus Deliverable Requirement

> *"*"Provisioned node, idempotency evidence, health-check result and rollback/recovery demonstration."*"*

---

3. Step-by-Step Guide: What to Show the Teacher

Step 1: Explain Idempotency & Rollback to Teacher

- **Speech Script:**

> *"*"Madam/Sir, in Task 14, I validated two foundational DevOps reliability principles: **Idempotency** and **Automated Rollback**. I proved idempotency by rerunning our Ansible playbook against the already-provisioned node; the re-run made zero redundant modifications, reporting `changed=0`. I also created an automated rollback playbook `rollback.yml` that reverts the deployment back to previous stable release `v1.0.0` within 8 seconds if a post-deployment health check fails."*"*

Step 2: Show the Idempotency Proof (`changed=0`)

Show the second execution log in [`docs/TASK-14-Automated-Provisioning-and-Reliability.md`](file:///c:/Users/ishwar/Desktop/devops%20gayatri/docs/TASK-14-Automated-Provisioning-and-Reliability.md):

```
PLAY RECAP *********************************************************************************************************
target-node-01             : ok=14   changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

- **Explain to Teacher:** *"Look at `changed=0`. This proves that Ansible checked all packages, users, folders, and systemd units, found them already in the desired state, and made zero redundant changes."*

Step 3: Show the Automated Health Check Script

Open file: [`ansible/healthcheck.sh`](file:///c:/Users/ishwar/Desktop/devops%20gayatri/ansible/healthcheck.sh)

Show how it probes the application endpoint:

```
HTTP_STATUS=$(curl -s -o /dev/null -w "%{http_code}" "$ENDPOINT")
if [ "$HTTP_STATUS" -eq 200 ]; then
    echo "[HEALTH CHECK PASSED] HTTP 200 OK"
    exit 0
else
    echo "[HEALTH CHECK FAILED] Expected HTTP 200, Received HTTP $HTTP_STATUS"
    exit 1
fi
```

Step 4: Show the Automated Rollback Playbook

Open file: [`ansible/rollback.yml`](file:///c:/Users/ishwar/Desktop/devops%20gayatri/ansible/rollback.yml)

Explain the 5 rollback steps:

1. Verify previous stable release `vaccination-reminder-portal-1.0.0.jar` exists in `/opt/vaccination-portal/releases/`.
2. Atomically re-point the symlink `/opt/vaccination-portal/app.jar` to `1.0.0.jar`.
3. Restart `vaxguard.service` via systemd.
4. Perform health check against `http://localhost:8080/api/health` until HTTP 200 is verified.
5. Output successful recovery report.

---

4. Likely Viva Voce Questions & Answers for Task 14

Q1: What is Idempotency in DevOps automation?

Answer: Idempotency is the property where executing an operation multiple times produces the exact same result as executing it once, without causing configuration drift, side effects, or duplicate resources.

Q2: How does your deployment architecture facilitate fast rollbacks?

Answer: Rather than overwriting the executable file, each release is version-stamped in `/opt/vaccination-portal/releases/`. The active service runs via a symbolic link (`app.jar`). To roll back, Ansible simply updates the symlink pointer to the previous version and restarts the service, completing rollback in under 8 seconds.

Q3: What triggers an automated rollback in a CI/CD pipeline?

Answer: If a post-deployment health check (`/api/health`) or automated smoke test fails to return HTTP 200 within a defined retry window, the pipeline automatically invokes the rollback playbook to prevent user-facing downtime.

Task 15: Final Release, Complete Documentation, and Viva Voce

DevOps Laboratory Assignment & Viva Voce Document

Class / Semester: TE Computer Engineering / DevOps Lab

Project Title: Selenium Testing Pipeline for a Vaccination Reminder Portal (VaxGuard)

---

1. Aim of the Experiment

To execute the complete end-to-end Vaccination Reminder Portal DevOps workflow (from Git commit to Jenkins build, Selenium automated testing quality gate, Docker containerization, and Ansible server provisioning), assemble complete technical documentation, architecture diagrams, and troubleshooting guides, identify limitations and future enhancements, and successfully defend the project in the Viva Voce examination.

---

2. Syllabus Deliverable Requirement

> *"*"Final repository, live demonstration, complete report, screenshots/video, presentation and viva."*"*

---

3. Step-by-Step Guide: What to Show the Teacher

Step 1: Explain the Final Release to Teacher

- **Speech Script:**

> *"*"Madam/Sir, Task 15 represents the culmination of all 15 stages into our release baseline `v1.0.0`. I have verified the complete pipeline from Git commit, through Maven build, Selenium quality gate, Docker containerization, and Ansible server provisioning. All 15 individual deliverables are documented in our repository. Now I would like to present a live 2-minute demonstration of VaxGuard."*"*

Step 2: Perform the 2-Minute Live Demo (on `http://localhost:8080`)

1. **Show the Summary Dashboard:**
   - Point to the KPI counters: *Registered Children (4), Doses Administered (17), Upcoming (7 Days), Overdue Doses (22), Compliance Rate (23.6%)*.
   - Point to the interactive **Chart.js** doughnut chart breaking down completed, scheduled, and overdue doses.
2. **Demonstrate Child Registration & Automated Schedule Engine:**
   - Click the blue button: `[■ Register Beneficiary]`.
   - Fill details: Child Name *"Vihaan Patil"*, DOB *today minus 8 weeks*, Gender *Male*, Parent Name *Rohan Patil*, Phone *+91 98200 12345*.
   - Click `[Save & Generate Schedule]`. Show the success toast!
3. **Show Vaccination Records Table:**
   - Click **"Vaccination Records"** tab.
   - Search *"Vihaan Patil"*. Show how all 16 vaccine doses (BCG, HepB, OPV-0, OPV-1, Pentavalent-1, Rotavirus-1, PCV-1, MR, DPT) were automatically calculated!
4. **Demonstrate Role-Based Clinical Administration:**
   - Click `[■ Administer]` on a scheduled dose.
   - Enter administering nurse name (*Staff Nurse Gayatri Patil*) and batch lot number (*LOT-2026-SERUM*).
   - Click Confirm. Show how the status badge changes to green **COMPLETED** with clinical administration metadata recorded!
   - Switch to **Parent View** role: show how the administration buttons are hidden for parents.
5. **Demonstrate Smart Reminder Simulator:**
   - Click `[■ Dispatch Due Reminders]`. Show the simulated SMS and Email broadcast dispatched to parents!

Step 3: Show the Master README and Documentation Dossiers

Open file: [`README.md`](file:///c:/Users/ishwar/Desktop/devops%20gayatri/README.md)

Point to the 15-Deliverable Matrix showing all 15 tasks complete and linked.

---

4. Comprehensive Final Viva Voce Questions & Answers

Q1: What is the primary difference between Unit Testing and Selenium WebDriver Testing in DevOps?

Answer: Unit tests (like JUnit Mockito tests) verify isolated Java methods in memory without a web browser. Selenium WebDriver tests are end-to-end UI regression tests that control a real browser, click elements, fill forms, verify JavaScript events, and validate that the

backend, database, and frontend interact harmoniously.

Q2: What is a Quality Gate in Jenkins and why is it essential?

Answer: A Quality Gate is an automated threshold configured in the CI/CD pipeline. In our pipeline, if any Selenium test assertion fails, the build halts immediately, captures a failure screenshot into `target/screenshots/`, and skips packaging and Docker deployment to ensure broken code never reaches users.

Q3: How does your Docker setup support modern microservices best practices?

Answer: We implemented a multi-stage Dockerfile using Eclipse Temurin JDK 21 for building and Alpine JRE 21 for execution, shrinking image size to 214MB. We enforced container security by running under an unprivileged user (`vaxuser:vaxgroup`) and configured container health checks (`/api/health`).

Q4: What is Idempotency in Ansible and how did you prove it?

Answer: Idempotency guarantees that executing an automation playbook multiple times leaves the system in the exact same state without unintended side effects. We proved it by running `ansible/playbook.yml` twice: the first run made 12 changes, and the second run reported `changed=0`, proving complete idempotency.

Q5: How does your rollback mechanism work?

Answer: We developed `ansible/rollback.yml`. Application versions are preserved in `/opt/vaccination-portal/releases/`. If an active release fails health checks, Ansible atomically re-points the symbolic link `app.jar` back to `vaccination-reminder-portal-1.0.0.jar`, restarts the systemd service, and verifies the `/api/health` probe within 8 seconds.